

RAPPORT DE PROJET

Projet de Fin de Module — Deep Learning

Conception, Implémentation et Analyse Critique de Modèles Deep Learning pour Données Tabulaires, Images et Séquences

MLP • CNN • RNN • LSTM • GRU • Seq2Seq • Interface Streamlit

Réalisé par :	REGRAGUI Nisrine
Encadrée par :	Mme. HIDILA Zineb
Filière / Groupe :	Informatique — 4IADA, Groupe 2
Année universitaire :	2025 – 2026
Établissement :	EMSI Casablanca — Campus Anfa

Année Universitaire 2025 – 2026

Introduction

1.1 Contexte et problématique

Le deep learning constitue aujourd'hui le paradigme dominant de l'apprentissage automatique, capable de modéliser des relations complexes dans des données hétérogènes : tableaux structurés, images, séquences textuelles. Ce projet de fin de module s'inscrit dans cette dynamique en proposant une étude comparative rigoureuse de trois familles d'architectures fondamentales, appliquées à des jeux de données réels dans le domaine médical et linguistique.

La problématique centrale de ce travail est la suivante : comment le deep learning adapte-t-il ses architectures à la structure intrinsèque des données — tabulaire, image, séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

1.2 Objectifs du projet

- Maîtriser la construction de modèles PyTorch avec `nn.Module`, `nn.Sequential` et les classes personnalisées.
- Implémenter, entraîner et comparer un MLP sur données tabulaires (Partie I).
- Concevoir un CNN inspiré de LeNet pour la classification d'images médicales (Partie II).
- Construire et évaluer un pipeline RNN / LSTM / GRU et un système Seq2Seq pour la traduction automatique (Partie III).
- Développer une interface de déploiement Streamlit intégrant les modèles entraînés.
- Produire une analyse critique transversale reliant théorie, choix architecturaux et résultats expérimentaux.

1.3 Vue d'ensemble des réalisations

Le tableau ci-dessous synthétise l'ensemble des travaux réalisés dans le cadre de ce projet.

Architecture	Données	Dataset utilisé	Performance
MLP	Données tabulaires	Breast Cancer Wisconsin	Accuracy : 98.8%
CNN (LeNet+FlexCNN)	Images médicales	PneumoniaMNIST (28×28)	Accuracy : 88.5%, F1 : 0.91
RNN / LSTM / GRU	Séquences textuelles	Tatoeba fra-eng (20k paires)	GRU Val PPL : 3.34
Seq2Seq (LSTM)	Traduction fr→en	Tatoeba fra-eng	BLEU Beam : 0.5531
Interface Streamlit	Déploiement	MLP + FlexCNN	2 modèles déployés

Tableau 1 — Synthèse des architectures, datasets et performances obtenues.

1.4 Environnement technique

- **Langage** : Python 3.10
- **Framework DL** : PyTorch 2.x
- **Datasets** : scikit-learn (Breast Cancer Wisconsin), MedMNIST (PneumoniaMNIST), Tatoeba fra-eng (Anki)
- **Déploiement** : Streamlit — interface web interactive
- **Reproductibilité** : SEED = 42 fixé (torch, numpy, random) dans les trois parties
- **Hardware** : CPU (contrainte de ressources) — gestion device CPU/GPU implémentée

Partie I — MLP et Ingénierie PyTorch

Dataset : Breast Cancer Wisconsin

569 échantillons — 30 features cliniques (Radius, Texture, Perimeter, Area, Smoothness...)

2 classes : Malin (212 cas) / Bénin (357 cas) — ratio déséquilibre 1:1.68

Aucune valeur manquante — données déjà propres

Source : `sklearn.datasets.load_breast_cancer()`

1.1 Fondements théoriques

1.1.1 nn.Module — La brique fondamentale de PyTorch

Tout modèle PyTorch hérite de la classe `nn.Module`, qui expose deux mécanismes essentiels : la gestion automatique des paramètres apprenables (poids et biais) via le graphe de calcul, et le dispatch de la propagation avant (forward) lors de chaque appel au modèle. La méthode `forward()` doit être obligatoirement surchargée et définit le calcul à effectuer.

Propagation avant : Pour un neurone, la sortie est $y = f(W^T x + b)$ où W est la matrice de poids, x le vecteur d'entrée, b le biais et f la fonction d'activation. Pour le réseau entier, chaque couche applique cette transformation successivement.

Rétropropagation : L'algorithme de rétropropagation calcule les gradients de la loss par rapport à chaque paramètre via la règle de la chaîne (chain rule). PyTorch construit automatiquement le graphe de calcul lors du forward et calcule les gradients lors de l'appel à `loss.backward()`. Le mécanisme autograd rend ce processus transparent.

1.1.2 state_dict() et named_parameters()

La méthode `state_dict()` retourne un dictionnaire ordonné associant le nom de chaque paramètre à son tenseur. Elle est utilisée pour la sauvegarde et le rechargement des modèles. La méthode `named_parameters()` itère sur tous les paramètres avec leur nom hiérarchique, permettant une inspection fine des couches.

1.2 Préparation des données

- **Split stratifié** : 70% entraînement (398), 15% validation (85), 15% test (86) — `stratify=y` pour préserver la distribution des classes.
- **StandardScaler** : Fitté exclusivement sur le train set, puis appliqué en transform sur val et test. Préviend le data leakage.
- **TensorDataset + DataLoader** : Batch size 32, `shuffle=True` pour le train uniquement. Conversion en `dtype=torch.float32` pour les features, `torch.long` pour les labels.

1.3 Implémentation des deux versions MLP

1.3.1 Version nn.Sequential

La version Sequential enchaîne les couches de manière linéaire : `Linear(30→64) → ReLU → Dropout(0.3) → Linear(64→32) → ReLU → Dropout(0.3) → Linear(32→2)`. Simple et lisible, elle convient aux architectures sans branchement.

1.3.2 Version Classe Personnalisée

La classe MLP hérite de `nn.Module` et accepte une liste d'architectures `hidden_dims`, permettant une flexibilité totale. Son `forward()` délègue au `Sequential` interne. Cette approche est préférable dès qu'on a besoin d'accéder aux activations intermédiaires, d'implémenter des connexions résiduelles ou de personnaliser la propagation.

Couche	Type	Dimensions	Rôle
Entrée	Linear	30 → 64	Projection des features cliniques
Cachée 1	Linear+ReLU+Dropout(0.3)	64 → 32	Extraction de représentations
Cachée 2	Linear+ReLU+Dropout(0.3)	32 → 32	Raffinement des features
Sortie	Linear + Softmax	32 → 2	Probabilité Bénin/Malin

Tableau 2 — Architecture MLP classe personnalisée utilisée (`input_dim=30`, `hidden_dims=[64,32]`, `output_dim=2`).

1.4 Inspection des paramètres

L'inspection via `named_parameters()` révèle 8 tenseurs apprenables pour un total de 2 882 paramètres. Le premier tenseur de poids (`network.0.weight`, shape [64, 30]) présente après initialisation Xavier une moyenne proche de 0 et un écart-type de 0.16, cohérent avec la formule Xavier : $\text{std} = \sqrt{2 / (\text{fan_in} + \text{fan_out})}$.

1.5 Stratégies d'initialisation

- **Gaussienne (std=0.01)** : Poids très petits → gradient vanishing dès les premières couches → convergence lente. Accuracy finale : 96.5%.
- **Constante (0.1)** : Symétrie parfaite → tous les neurones évoluent de la même manière → pas de spécialisation. Accuracy finale : 94.1%.
- **Xavier Uniform ✓** : Préserve la variance des activations et des gradients à travers les couches. Meilleure convergence et accuracy finale : 98.8%.

1.6 Entraînement et résultats

Optimiseur Adam ($\text{lr}=1\text{e-}3$), `CrossEntropyLoss`, 50 époques. Le meilleur modèle (val accuracy maximale) est sauvegardé via `torch.save(model.state_dict(), 'best_model.pth')` puis rechargé pour l'évaluation finale sur le test set.

Modèle	Accuracy	Precision	Recall	F1-Score
MLP nn.Sequential	97.4%	97.6%	97.4%	97.4%
MLP Classe Perso.	98.8%	98.9%	98.8%	98.8%
Init. Gaussienne	96.5%	96.7%	96.5%	96.5%
Init. Constante	94.1%	94.3%	94.1%	94.0%
Init. Xavier ✓	98.8%	98.9%	98.8%	98.8%

Tableau 3 — Comparaison des configurations MLP. Le modèle classe personnalisée avec initialisation Xavier atteint 98.8% d'accuracy.

1.7 Question de synthèse — Partie I

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel ?

Le MLP avec initialisation Xavier, régularisation Dropout(0.3) et optimiseur Adam atteint 98.8% d'accuracy sur le Breast Cancer Wisconsin Dataset, surpassant de nombreux modèles classiques (SVM, Random Forest typiquement entre 96-98%). Cette performance s'explique par la nature du dataset : 30 features continues, bien normalisées, sans structure spatiale ni temporelle, qui correspondent exactement au biais inductif du MLP (fully connected, invariant aux permutations de features).

Limites observées : (1) Le MLP est sensible à l'initialisation — une initialisation constante dégrade de 4.7 points. (2) Il n'exploite pas les corrélations structurées entre features (contrairement à un CNN sur images). (3) Sur des datasets tabulaires avec variables catégorielles ou manquantes, le prétraitement devient critique. (4) L'interprétabilité reste limitée sans méthodes supplémentaires (SHAP, feature importance). En conclusion, le MLP est optimal pour les données tabulaires bien préparées, mais ses limites émergent dès que les données ont une structure géométrique ou temporelle explicite.

Partie II — CNN et Vision par Ordinateur

Dataset : PneumoniaMNIST (MedMNIST)

Images de radiographies thoraciques 28×28 pixels, 1 canal (niveaux de gris)

Train : 4 708 images | Val : 524 | Test : 624

Classes déséquilibrées : Normal (1 214) vs Pneumonie (3 494) — ratio 1:2.88

Gestion du déséquilibre : class_weights calculés et intégrés dans CrossEntropyLoss

2.1 Pourquoi le MLP est inadapté aux images

Un MLP appliqué à une image 28×28 aplatit le tenseur en un vecteur de 784 éléments, perdant toute information spatiale. Deux pixels voisins dans l'image peuvent se retrouver éloignés dans ce vecteur, rendant impossible l'apprentissage de relations locales (bords, textures, formes). De plus, un MLP sur images ne bénéficie pas de l'invariance par translation : un objet décalé de quelques pixels sera traité comme un objet entièrement différent.

Les trois principes fondateurs des CNN : (1) **Localité** — les filtres convolutifs opèrent sur une fenêtre spatiale limitée (3×3, 5×5), capturant des patterns locaux. (2) **Partage de poids** — le même filtre est appliqué à toutes les positions spatiales, réduisant drastiquement le nombre de paramètres. (3) **Hiérarchie des représentations** — les premières couches détectent des bords et textures, les couches profondes des formes complexes.

2.2 Calculs manuels — Corrélation croisée 2D

La corrélation croisée 2D est l'opération fondamentale des CNN. Pour une entrée X de taille N×N et un noyau K de taille F×F avec padding P et stride S, la taille de sortie est :

$$\text{output_size} = \lfloor (N + 2P - F) / S \rfloor + 1$$

Application sur notre architecture LeNet (entrée 28×28) :

- **Conv1 (F=5, P=2, S=1) :** $\lfloor (28+4-5)/1 \rfloor + 1 = 28 \times 28 \rightarrow \text{MaxPool}(2,2) \rightarrow 14 \times 14$
- **Conv2 (F=5, P=0, S=1) :** $\lfloor (14-5)/1 \rfloor + 1 = 10 \times 10 \rightarrow \text{MaxPool}(2,2) \rightarrow 5 \times 5$
- **Conv3 1×1 (F=1, P=0, S=1) :** 5×5 → inchangé — recombine les canaux
- **Flatten :** 5×5×32 = 800 features → connecté au classifieur

Les implémentations manuelles de `cross_correlation_2d()`, `manual_max_pool()` et `manual_avg_pool()` ont été validées par comparaison avec les couches PyTorch correspondantes (différence maximale < 1e-6).

2.3 Architecture LeNet et FlexCNN

Deux architectures CNN ont été implémentées. Le LeNetCNN constitue la baseline inspirée de l'architecture de LeCun (1998) adaptée aux images 28×28 monoscale. Le FlexCNN est une version paramétrable permettant l'exploration systématique des choix architecturaux.

Bloc	Opérations	Sortie (exemple)	Remarques
Bloc 1	Conv2d(1,16,5,P=2)+BN+ReLU+MaxPool(2)	16×14×14	Padding same, détection bas niveau
Bloc 2	Conv2d(16,32,5,P=0)+BN+ReLU+MaxPool(2)	32×5×5	Réduction spatiale, patterns complexes
Bloc 3 (1×1)	Conv2d(32,32,1)+ReLU	32×5×5	Recombinaison des canaux (bottleneck)
Classif.	Flatten+Linear(800,120)+ReLU+Dropout(0.4)+Linear(120,84)+ReLU+Linear(84,2)	2	Couches denses finales

Tableau 4 — Architecture LeNetCNN déployée. La convolution 1×1 (Bloc 3) agit comme bottleneck de recombinaison des canaux.

La BatchNormalization après chaque convolution stabilise l'entraînement en normalisant les activations, ce qui accélère la convergence et réduit la sensibilité au learning rate. Le Dropout(0.4) dans le classifieur prévient le surapprentissage.

2.4 Étude expérimentale des choix architecturaux

Cinq configurations ont été systématiquement comparées en gardant tous les autres hyperparamètres identiques (lr=1e-3, Adam, 10 époques, SEED=42) :

Configuration	Accuracy	F1-Score	Paramètres	Observation
LeNet Baseline	86.2%	0.876	120 854	Référence — bonne baseline
Sans conv 1×1	85.8%	0.871	118 562	Légère dégradation — 1×1 utile
AvgPool (vs Max)	84.7%	0.860	120 854	MaxPool meilleur sur X-rays
Sans padding	83.1%	0.844	108 242	Perte d'info aux bords
More_filters (64) ✓	88.5%	0.910	191 102	Meilleure config — choix final

Tableau 5 — Résultats de l'étude expérimentale. La configuration More_filters (64 filtres) est retenue comme meilleur modèle.

Analyse des résultats : (1) **MaxPool vs AvgPool** : le MaxPool conserve mieux les pics d'activation (bords de lésions sur X-rays), d'où sa supériorité de +1.5 points. (2) **Padding** : son absence dégrade de 3.1 points en perdant l'information aux bords de l'image — critique sur des images médicales où les bords pulmonaires sont diagnostiques. (3) **Conv 1×1** : ajoute seulement 1K paramètres mais améliore de 0.4 points en permettant une recombinaison non-linéaire des canaux. (4) **More_filters** : doubler le nombre de filtres améliore significativement

(+2.3 pts) — le modèle dispose de plus de filtres pour capturer la diversité des patterns pneumoniques.

2.5 Visualisation des cartes de caractéristiques

Les feature maps extraites sur des images Normal vs Pneumonie montrent une progression cohérente : la couche Conv1 détecte des bords et gradients locaux, la couche Conv2 des textures plus complexes (patterns alvéolaires), et la couche Conv3 (1×1) des représentations sémantiques de plus haut niveau. Les cartes de la classe Pneumonie montrent systématiquement des activations plus intenses dans les régions centrales de l'image, correspondant aux zones d'infiltration pulmonaire.

2.6 Comparaison MLP vs CNN

MLP Baseline vs LeNet CNN vs FlexCNN (More_filters)

MLP Baseline : Accuracy 83.1% | F1 0.844 | 234 114 paramètres

LeNet CNN : Accuracy 86.2% | F1 0.876 | 120 854 paramètres

FlexCNN More_filters : Accuracy 88.5% | F1 0.910 | 191 102 paramètres

Le CNN atteint 88.5% avec 23% MOINS de paramètres que le MLP (191k vs 234k), démontrant l'efficacité de l'inductive bias spatial.

2.7 Question de synthèse — Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images médicales ?

Le CNN surpasse le MLP de 5.4 points d'accuracy (88.5% vs 83.1%) avec 19% moins de paramètres, validant expérimentalement l'adéquation entre l'inductive bias spatial du CNN et la structure des images. Sur PneumoniaMNIST, les patterns discriminants (infiltrats, consolidations) sont des patterns locaux spatialement cohérents — exactement ce que les filtres convolutifs sont conçus à détecter.

L'étude des hyperparamètres révèle que le padding est critique (−3.1 pts sans padding) car les bords pulmonaires sont diagnostiquement importants, que le MaxPool préserve mieux les features médicales que l'AvgPool (−1.5 pts), et que doubler les filtres améliore significativement (+2.3 pts) en augmentant la capacité de représentation. Ces résultats soulignent que les choix architecturaux doivent être guidés par la sémantique du domaine, pas uniquement par des heuristiques générales.

Partie III — RNN, LSTM, GRU et Seq2Seq

Dataset : Tatoeba / Anki fra-eng

Corpus parallèle français-anglais — source : www.manythings.org/anki/

Après filtrage (max 12 tokens) : 20 000 paires retenues

Split 80/10/10 : 16 000 train | 2 000 val | 2 000 test

Vocabulaire français : 5 947 tokens | Anglais : 3 286 tokens

Tokens spéciaux : PAD=0, SOS=1, EOS=2

3.1 Fondements théoriques

3.1.1 Modèle de langage et factorisation par la règle de chaîne

Un modèle de langage estime la probabilité d'une séquence de tokens. Par la règle de chaîne des probabilités, la probabilité d'une séquence ($w_1, w_2, \dots, w_t$) se factorise en :

$$P(w_1, \dots, w_t) = \prod_i P(w_i | w_1, \dots, w_{i-1})$$

Cette factorisation justifie l'approche autorégressive des modèles récurrents : à chaque pas de temps t , le modèle prédit le token suivant en conditionnant sur tous les tokens précédents, encodés dans l'état caché h_t .

3.1.2 Perplexité

Perplexité = exp(Cross-Entropy Loss). Elle mesure l'incertitude moyenne du modèle à chaque pas de temps — intuitivement, le nombre de tokens candidats effectifs à chaque prédiction. Un modèle parfait aurait une perplexité de 1 (certitude totale). Un modèle aléatoire sur un vocabulaire de 3 286 tokens aurait une perplexité de 3 286. Nos modèles atteignent des perplexités entre 3.34 (GRU) et 5.16 (RNN), soit une réduction d'un facteur ~638 par rapport au hasard.

3.1.3 Architectures récurrentes

RNN simple : $h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$. L'état caché accumule l'information des tokens passés. Le problème majeur est le gradient évanescent lors de la rétropropagation à travers le temps (BPTT) : les gradients diminuent exponentiellement avec la distance temporelle, limitant la mémoire effective à ~7 tokens.

LSTM : Ajoute une cellule mémoire c_t indépendante de h_t . Trois portes contrôlent les flux : porte d'oubli (f_t), porte d'entrée (i_t) et porte de sortie (o_t). La mise à jour additive de c_t crée une autoroute pour les gradients, permettant de mémoriser des dépendances très longues.

GRU : Simplifie le LSTM en fusionnant les portes d'oubli et d'entrée en une porte d'update, et en supprimant la cellule mémoire séparée. Moins de paramètres, convergence plus rapide, performances comparables au LSTM sur séquences courtes à moyennes.

3.2 Préparation des données

- **Normalisation** : NFD Unicode, mise en minuscules, espace avant ponctuation, suppression des caractères non-alphabétiques.

- **Filtrage** : MAX_LEN=12 tokens pour contrainte CPU — exclut les phrases longues (réduit de ~220k à 20k paires).
- **Encodage** : Chaque token converti en indice entier, padding à MAX_LEN+1 avec token PAD=0.
- **Masquage** : CrossEntropyLoss avec ignore_index=PAD — les tokens de padding n'influencent pas la loss.
- **DataLoader** : Batch size 64, shuffle=True pour train.

3.3 Comparaison RNN / LSTM / GRU

Les trois modèles partagent les mêmes hyperparamètres (embed_dim=128, hidden_dim=256, n_layers=2, dropout=0.3, lr=1e-3, 10 époques) pour une comparaison équitable. Le seed est réinitialisé (SEED=42) avant chaque entraînement.

Modèle	Paramètres	Val Loss	Val PPL	Stabilité	Mémorisation
RNN	2.61M	1.64	5.16	Faible	Limitée (≤ 7 tokens)
LSTM	3.21M	1.32	3.74	Bonne	Excellente
GRU ✓	2.98M	1.21	3.34	Très bonne	Très bonne

Tableau 6 — Comparaison RNN / LSTM / GRU. Le GRU offre le meilleur compromis perplexité / nombre de paramètres.

3.4 Illustration expérimentale du Gradient Clipping

Une expérience dédiée compare l'entraînement d'un VanillaRNN avec et sans gradient clipping (max_norm=1.0) sur 3 époques, 30 batches chacune :

- **Sans clipping** : Norme maximale des gradients : ~4.1 (pic au batch 40). Instabilité de la loss visible.
- **Avec clipping** : Norme bornée à 1.0 à chaque batch. Loss plus régulière et convergence plus stable.

Sur séquences courtes (12 tokens), l'impact est préventif. Sur des séquences longues, un seul gradient explosif peut corrompre des dizaines d'époques d'entraînement. Le clip_grad_norm_(max_norm=1.0) est donc une bonne pratique systématique pour les architectures récurrentes.

3.5 Système Seq2Seq — Encodeur-Décodeur

Un mini-système de traduction automatique français→anglais a été construit autour d'une architecture encodeur-décodeur LSTM. L'encodeur lit la phrase source et compress son contenu dans un vecteur de contexte (hidden state final). Le décodeur génère la traduction token par token en conditionnant sur ce vecteur.

- **Teacher Forcing (ratio=0.5)** : À chaque pas de décodage, le token précédent utilisé est tiré aléatoirement entre le token correct (teacher) et la prédiction du modèle. Ratio 0.5 = équilibre entre stabilité d'entraînement et apprentissage de la correction d'erreurs.
- **Checkpoint** : Le meilleur état (min val_loss) est sauvegardé et rechargé pour l'évaluation finale.

- **Résultats** : Val loss ~ 1.10 à partir de l'époque 7, légère divergence train/val indiquant le début d'overfitting — attendu sans mécanisme d'attention sur 20k paires.

3.6 Stratégies de décodage et évaluation BLEU

Deux stratégies de décodage ont été implémentées et évaluées sur 200 paires de test :

- **Greedy** : argmax à chaque pas $\rightarrow O(T)$ opérations, localement optimal. Rapide mais un mauvais choix à t compromet tous les pas suivants.
- **Beam Search ($k=3$)** : Maintient les 3 séquences candidates les plus probables. Scores en log-probabilités normalisés par la longueur ($\alpha=0.7$) pour éviter de pénaliser les traductions longues.

Stratégie	BLEU	BLEU-1	BLEU-2	Observation
Greedy decode	0.4747	0.68	0.52	Rapide, localement optimal
Beam Search $k=3$ ✓	0.5531	0.74	0.61	+16.5% vs greedy — meilleur global

Tableau 7 — Comparaison Greedy vs Beam Search. Le Beam Search améliore le score BLEU de +16.5%.

3.7 Question de synthèse — Partie III

Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle ?

Le passage RNN $\rightarrow$ LSTM/GRU $\rightarrow$ Seq2Seq répond à des limitations progressivement identifiées. Le RNN simple (PPL=5.16) souffre du gradient évanescent qui limite sa mémoire effective à ~ 7 tokens, insuffisante pour des phrases de 12 tokens en moyenne. Le LSTM (PPL=3.74) résout ce problème via la cellule mémoire additive, mais au prix d'un surcoût paramétrique (+23%). Le GRU (PPL=3.34) offre un meilleur compromis — performances supérieures avec 7% moins de paramètres que le LSTM.

Pour la traduction, les modèles LM mono-séquence sont insuffisants : ils ne peuvent pas conditionner la génération sur une phrase source. L'architecture Seq2Seq répond à ce besoin en séparant l'encodage (compréhension de la source) du décodage (génération de la cible). Le Beam Search améliore de +16.5% le BLEU en explorant un espace de séquences plus large.

Limite fondamentale observée : le bottleneck du vecteur de contexte fixe. Sur des phrases longues, l'encodeur doit compresser toute l'information source en un unique vecteur — perte d'information inévitable. La solution naturelle est le mécanisme d'attention (Bahdanau 2015), qui permet au décodeur de consulter directement les états cachés de l'encodeur à chaque pas — fondement des Transformers.

Interface de Déploiement — Streamlit

En complément des trois parties expérimentales, une interface web interactive a été développée avec Streamlit pour déployer les modèles MLP et FlexCNN (More_filters) dans un environnement accessible aux utilisateurs non-techniques, notamment dans un contexte médical d'aide à la décision.

4.1 Architecture de l'interface

4.1.1 Navigation (Sidebar)

La barre latérale propose un menu de navigation multi-rubriques : Dashboard principal, Breast Cancer Classification, Pneumonia Detection, Model Performance et About. Une section Tech Stack affiche les technologies utilisées (Python, PyTorch, scikit-learn, Streamlit).

4.1.2 Dashboard — Métriques globales

Le haut du Dashboard affiche 4 KPIs (Key Performance Indicators) synthétiques : 2 modèles déployés, 98.8% accuracy cancer du sein, 88.5% accuracy pneumonie, 0.91 F1-Score pneumonie.

4.2 Panneau MLP — Classification du Cancer du Sein

- **Onglet Manual Input** : Formulaire interactif pour les 30 features cliniques du Wisconsin Dataset (Radius Mean, Texture Mean, Perimeter Mean, Area Mean, Smoothness Mean, etc.) avec sliders calibrés sur les distributions du dataset.
- **Onglet CSV Upload** : Glisser-déposer un fichier CSV contenant les features d'un ou plusieurs patients — prédiction en batch.
- **Résultat** : Affichage de la prédiction (Bénin/Malin) avec un graphique donut interactif Plotly montrant le pourcentage de confiance pour chaque classe.
- **Architecture visuelle** : Schéma Input → ReLU+Dropout → Output représentant les couches du MLP.

4.3 Panneau CNN — Détection de la Pneumonie

- **Upload d'image** : Interface drag-and-drop pour radiographies thoraciques (JPG/PNG).
- **Prétraitement automatique** : Redimensionnement en 28×28, conversion niveaux de gris, normalisation (mean=0.5, std=0.5), conversion en tensor PyTorch.
- **Prévisualisation** : L'image uploadée est affichée avec indication de redimensionnement.
- **Résultat** : Prédiction (Normal/Pneumonie) avec donut chart rouge/vert et score de confiance.
- **Architecture visuelle** : Schéma Conv2D → BatchNorm → MaxPool → ... représentant les blocs FlexCNN.

4.4 Disclaimer médical

Clause de non-responsabilité

L'application a été développée dans un cadre universitaire à des fins d'apprentissage et de démonstration.
 Les prédictions produites par les modèles ne remplacent en aucun cas un avis médical professionnel.
 Elles ne doivent pas être utilisées pour des décisions cliniques réelles sans validation par un médecin qualifié.

4.5 Tableau récapitulatif des fonctionnalités

Composant	Modèle MLP	Modèle CNN (FlexCNN)
Dataset	Breast Cancer Wisconsin	PneumoniaMNIST
Input	30 features cliniques (formulaire / CSV)	Image radiologique JPG/PNG
Preprocessing	StandardScaler	Redimensionnement 28×28 niveaux de gris
Output	Bénin / Malin + donut chart confiance	Pneumonie / Normal + donut chart
Performance	Accuracy : 98.8%	Accuracy : 88.5%, F1 : 0.91

Tableau 8 — Comparaison des fonctionnalités des deux panneaux de l'interface Streamlit.

Discussion Transversale

Problématique : Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

5.1 Adéquation entre structure des données et biais inductif

Le résultat fondamental de ce projet est que la performance d'un modèle deep learning est déterminée non seulement par sa capacité paramétrique, mais par l'adéquation entre son biais inductif et la structure intrinsèque des données. Un biais inductif est l'ensemble des hypothèses implicites qu'une architecture fait sur les données : le MLP suppose que toutes les features contribuent de manière indépendante, le CNN suppose une structure spatiale locale et invariante par translation, les architectures récurrentes supposent une dépendance séquentielle temporelle.

Sur le Breast Cancer Wisconsin, le MLP atteint 98.8% précisément parce que les 30 features cliniques sont des mesures statistiques indépendantes sans structure spatiale ou temporelle — l'absence de biais inductif est ici un avantage. Appliquer un CNN à ces données serait inapproprié car il n'existe pas de voisinage spatial entre les features.

Sur PneumoniaMNIST, le CNN dépasse le MLP de 5.4 points avec 23% moins de paramètres. L'invariance par translation et la détection de patterns locaux (bords pulmonaires, infiltrats) sont précisément les propriétés diagnostiques recherchées.

Sur le corpus Tatoeba, ni le MLP ni le CNN ne seraient applicables sans transformation préalable majeure — la dépendance temporelle entre tokens est constitutive de la sémantique linguistique.

5.2 Convergence des bonnes pratiques

Malgré leurs différences architecturales, les trois parties convergent vers les mêmes pratiques fondamentales :

- **Régularisation systématique :** Dropout (0.3-0.4) dans les trois architectures, BatchNorm dans le CNN.
- **Adam optimizer :** $lr=1e-3$, convergence robuste dans les trois cas.
- **Reproductibilité :** SEED=42 partout — condition sine qua non de la comparabilité expérimentale.
- **Initialisation Xavier :** Prouvée supérieure dans la Partie I (+4.7 pts vs constante), recommandée dans toutes les parties.
- **Gradient clipping :** Spécifique aux RNN, mais illustre un problème général de la stabilité d'entraînement.
- **Sauvegarde du meilleur modèle :** val_loss ou val_accuracy comme critère — protège contre l'overfitting final.

5.3 Limites et perspectives

- **MLP :** Pas d'interprétabilité native — SHAP ou permutation feature importance seraient nécessaires pour un usage clinique réel.

- **CNN** : L'absence de Grad-CAM ou de saliency maps limite l'explicabilité — critique pour la validation médicale des prédictions.
- **Seq2Seq** : Le bottleneck du vecteur de contexte fixe est la limitation fondamentale. L'attention de Bahdanau (2015) puis les Transformers (Vaswani 2017) ont résolu ce problème — conduisant aux LLMs actuels.
- **Interface Streamlit** : Déploiement local uniquement. Un déploiement cloud (Streamlit Cloud, Hugging Face Spaces) permettrait un accès élargi.
- **Perspective architecturale** : Vision Transformers (ViT) pour les images médicales, BERT/GPT pour les séquences — les architectures basées sur l'attention ont supplanté les CNN et RNN sur la plupart des benchmarks depuis 2020.

Conclusion Générale

Ce projet de fin de module a permis de mettre en oeuvre et d'analyser de manière critique les trois grandes familles d'architectures deep learning sur des données réelles à enjeux médicaux et linguistiques. Les résultats expérimentaux valident empiriquement le principe théorique central : l'efficacité d'un modèle deep learning est avant tout déterminée par l'adéquation entre son biais inductif et la structure des données.

Partie I — MLP : Un MLP bien paramétré (initialisation Xavier, Dropout, Adam) atteint 98.8% d'accuracy sur le Breast Cancer Wisconsin, démontrant qu'une architecture simple mais bien calibrée surpasse souvent des architectures complexes mal adaptées aux données.

Partie II — CNN : Le FlexCNN (More_filters) atteint 88.5% d'accuracy et 0.91 de F1 sur PneumoniaMNIST, avec 23% moins de paramètres que le MLP baseline. L'étude expérimentale systématique a montré l'impact décisif du padding (+3.1 pts), du type de pooling (+1.5 pts) et du nombre de filtres (+2.3 pts).

Partie III — Seq2Seq : Le GRU offre le meilleur compromis (PPL=3.34 avec 2.98M paramètres). Le système Seq2Seq LSTM atteint un BLEU de 0.5531 avec Beam Search (k=3), soit +16.5% vs greedy, validant l'intérêt de l'exploration de l'espace de séquences.

Interface Streamlit : Le déploiement des modèles MLP et FlexCNN dans une interface médicale interactive complète la chaîne de valeur du projet, de l'entraînement à l'utilisation clinique.

En termes de perspectives, les architectures Transformers (Vision Transformers pour les images, BERT/GPT pour les séquences) constituent la frontière actuelle du domaine — leur mécanisme d'attention généralisée surmonte les limitations identifiées dans ce travail (bottleneck Seq2Seq, absence de dépendances globales dans le CNN). Ce projet constitue donc un socle théorique et expérimental solide pour aborder ces architectures avancées.

Références Bibliographiques

[1]	Rosenblatt, F. (1958). The Perceptron. <i>Psychological Review</i> , 65(6), 386–408.
[2]	Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. <i>Nature</i> , 323, 533–536.
[3]	LeCun, Y., et al. (1998). Gradient-Based Learning Applied to Document Recognition. <i>Proceedings of the IEEE</i> , 86(11), 2278–2324.
[4]	Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. <i>Neural Computation</i> , 9(8), 1735–1780.
[5]	Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder. <i>EMNLP 2014</i> .
[6]	He, K., et al. (2016). Deep Residual Learning for Image Recognition. <i>CVPR 2016</i> .
[7]	Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. <i>ICLR 2015</i> .

[8]	Srivastava, N., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 15, 1929–1958.
[9]	Ioffe, S. & Szegedy, C. (2015). Batch Normalization. ICML 2015.
[10]	Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks. ICCV 2017.
[11]	Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
[12]	PyTorch Documentation officielle : https://pytorch.org/docs/stable/

Annexes

Annexe A — Structure des notebooks

- **Untitled1.ipynb** — Partie I : MLP sur Breast Cancer Wisconsin (PyTorch nn.Module, initialisations, sauvegarde)
- **Untitled13.ipynb** — Partie II : CNN sur PneumoniaMNIST (LeNetCNN, FlexCNN, feature maps)
- **part3_updated.ipynb** — Partie III : RNN/LSTM/GRU + Seq2Seq sur Tatoeba fra-eng (BLEU, Beam Search)

Annexe B — Hyperparamètres récapitulatifs

Hyperparamètre	MLP (Part I)	CNN (Part II)	RNN/Seq2Seq (Part III)
Optimizer	Adam	Adam	Adam
Learning rate	1e-3	1e-3	1e-3
Batch size	32	64	64
Epochs	50	15 (LM) / 15 (Seq2Seq)	10 (LM) / 15 (Seq2Seq)
Dropout	0.3	0.4 (classifier)	0.3
Grad clipping	—	—	max_norm=1.0
Seed	42	—	42
Loss	CrossEntropyLoss	CrossEntropyLoss+weights	CrossEntropyLoss+PAD mask

Tableau A1 — Récapitulatif des hyperparamètres utilisés dans les trois parties.